

Use Case 9: Continuous Deployment

Scenario: Set up a CI/CD pipeline.

Tasks:

- Use CodePipeline, CodeBuild, and CodeDeploy.
 - Deploy updates to an application hosted on EC2.
 - Rollback automatically on deployment failures.
-

Step 1: Set Up EC2 Environment

1. Launch EC2 Instances:
 - Go to the AWS Management Console and launch one or more EC2 instances.
 - Choose an Amazon Machine Image (AMI) that suits your application, such as Amazon Linux 2.

- Ensure the EC2 instances are in the same VPC and region.
- 2. Install Required Software on EC2:
 - Install dependencies required for your application, such as Python, Node.js, or Java.
 - Install and configure the CodeDeploy agent:

```
bash
```

```
CopyEdit
```

```
sudo yum update -y
```

```
sudo yum install ruby -y
```

```
sudo yum install wget -y
```

```
cd /home/ec2-user
```

```
wget https://bucket-  
name.s3.region.amazonaws.com/latest/in  
stall
```

```
chmod +x ./install
```

```
sudo ./install auto
```

```
sudo service codedeploy-agent start
```

3. Create an IAM Role for EC2:
 - Create an IAM Role with the AmazonEC2RoleforAWSCodeDeploy policy attached.
 - Attach this role to your EC2 instances.
4. Tag EC2 Instances:
 - Add a Tag to identify the EC2 instances to deploy to, e.g.:
 - Key: Name
 - Value: CodeDeployTarget

Step 2: Prepare Your Application

1. Package Your Application:

- Organize the application files into a structure that includes an appspec.yml file.
1. Upload to S3 or CodeCommit:
 - Use an S3 bucket or AWS CodeCommit repository to store your application code.
-

Step 3: Set Up CodePipeline

1. Create a Pipeline:
 - Go to AWS CodePipeline and click Create Pipeline.
 - Enter a Pipeline Name (e.g., MyAppPipeline).
 - Choose a New Service Role to allow CodePipeline to access other AWS services.
2. Add a Source Stage:

- Choose the source for your code:
 - S3: If your application is stored in an S3 bucket, specify the bucket name and object key.
 - CodeCommit: If using CodeCommit, select the repository and branch.
 - 3. Add a Build Stage:
 - Add AWS CodeBuild as the build provider.
 - Create a new CodeBuild project if not already set up.
-

Step 4: Configure CodeBuild

1. Create a CodeBuild Project:
 - In the CodeBuild Console, click Create Build Project.

- Enter a name for the project (e.g., MyAppBuild).
 - Specify the source as CodePipeline.
 - Choose a Build Environment:
 - Operating System: Amazon Linux 2.
 - Runtime: Standard (choose the latest runtime version).
 - 1. Test Your Build Project:
 - Run a test build to ensure CodeBuild works correctly.
-

Step 5: Configure CodeDeploy

1. Create an Application in CodeDeploy:
 - Go to AWS CodeDeploy Console and create a new application.

- Choose EC2/On-Premises as the compute platform.
 - Name your application (e.g., MyAppCodeDeploy).
2. Create a Deployment Group:
- Specify a Deployment Group Name.
 - Select your application.
 - In the Environment Configuration section, choose:
 - Amazon EC2 Instances.
 - Use the Tag Key-Value Pair to select your EC2 instances (e.g., Name=CodeDeployTarget).
 - Enable Rollback on Deployment Failures.
 - Specify a Service Role with the AWSCodeDeployRole policy.
3. Test CodeDeploy:

- Perform a manual deployment to ensure that CodeDeploy can successfully deploy to your EC2 instances.
-

Step 6: Complete the Pipeline

1. Add a Deploy Stage:
 - Return to your CodePipeline configuration and add a Deploy Stage.
 - Choose AWS CodeDeploy as the deploy provider.
 - Select the application and deployment group created earlier.
2. Save and Start the Pipeline:
 - Save your pipeline and release a change to test the end-to-end process.

Step 7: Enable Automatic Rollbacks

1. Enable Rollbacks in CodeDeploy:
 - Go to your CodeDeploy Deployment Group.
 - Under Deployment Settings, enable Rollback on Deployment Failures.
 - CodeDeploy will monitor health checks and automatically roll back if an instance fails.
2. Configure Health Checks:
 - Define Health Check Scripts in your `appspec.yml` to verify that the application is running correctly.

Step 8: Test the Pipeline

1. Make a Code Change:

- Push an update to your source repository or upload a new version to the S3 bucket.
 - 2. Observe the Pipeline Execution:
 - Monitor the progress in the CodePipeline Console.
 - Check the Source, Build, and Deploy stages.
 - 3. Simulate a Deployment Failure:
 - Introduce a failure (e.g., remove a required dependency) and observe the rollback process.
-

Step 9: Secure and Optimize

1. Secure IAM Roles:
 - Ensure that IAM roles have the principle of least privilege.

- Limit access to only the resources needed for each stage of the pipeline.
 - 2. Use AWS Secrets Manager:
 - Store sensitive information such as database credentials securely using Secrets Manager.
 - 3. Cost Optimization:
 - Monitor costs using AWS Cost Explorer.
 - Use reserved or spot instances for EC2 where applicable.
 - Terminate unused resources.
-

Step 10: Monitor and Maintain

1. Set Up CloudWatch Alarms:
 - Monitor the health of your EC2 instances and pipeline stages.

- Set up alarms for failed deployments or high latency.
2. Regularly Update Scripts and Configurations:
- Update the `appspec.yml` and build scripts to adapt to changes in your application or environment.